Construire le contenu du portfolio avec Supabase

I. Comprendre la base et créer le projet

Une table devient une route REST

Supabase héberge des serveurs PostgreSQL et expose les tables autorisées par une API REST.

Une table `articles` du schéma `public` correspond à `/rest/v1/articles`.

```
Navigateur -> API REST Supabase -> règles RLS -> PostgreSQL
```

Le tableau de bord sert à créer les tables et à saisir les contenus.

Documentation Supabase - Data REST API

RLS : contrôler les lignes accessibles

Row Level Security (RLS) désigne les règles d'accès aux enregistrements de PostgreSQL. Les droits SQL autorisent une opération sur une table. Une politique RLS décide quelles lignes le rôle peut atteindre.

```
articles : id=1, title="Publié", published_at="2026-09-24"
articles : id=2, title="Brouillon", published_at=NULL
GET /rest/v1/articles?select=id,title
Réponse pour anon : [{"id":1,"title":"Publié"}]
```

Le rôle `anon` possède le droit `SELECT`.

Une règle n'autorise que les lignes dont `published_at IS NOT NULL`. Le brouillon reste dans la base.

Documentation Supabase - Row Level Security

Configurer RLS dans Supabase

Dans **SQL Editor**, le script complet du cours configure `articles` ainsi :

```
alter table public.articles enable row level security;

revoke all on table public.articles from anon, authenticated;

grant usage on schema public to anon, authenticated;

grant select on table public.articles to anon, authenticated;

create policy "Read published articles"
  on public.articles for select
  to anon, authenticated
  using (published_at is not null);
```

`grant select` autorise la lecture pour tous les utilisateurs.

La "policy" masque les brouillons, c'est à dire les articles qui n'ont pas de date de publication.

Documentation Supabase - Configurer RLS

Créer une base personnelle

1. Ouvrir supabase.com/dashboard. Créer un compte si besoin.
2. Choisir une organisation, puis **New project**.
3. Nommer le projet comme bon vous semble, choisir une région proche.
4. Choisir un mot de passe fort pour la bdd.
5. Valider, puis attendre que le projet soit prêt.

Chaque projet possède sa propre base PostgreSQL et sa propre URL d'API.

- Coucou

Le mot de passe PostgreSQL ne sert pas aux requêtes REST du navigateur.

Documentation Supabase - Démarrer avec un projet

Prévoir les contenus du portfolio

Le portfolio est composé de 4 tables (libre à vous d'en créer plus !)

TABLE	USAGE	PUBLICATION
`articles`	billets et études de cas	
`pages`	présentation et parcours	
`projects`	réalisations	
`skills`	compétences affichées	

Un `published_at` vide signifie brouillon.

Un `timestamptz` renseigné signifie contenu publié.

Les images seront d'abord représentées par une URL `cover_url`.

Documentation Supabase - Types PostgreSQL

II. Création des tables

Exécuter un script unique

Pour s'éviter de créer chaque table à la main, nous allons utiliser un "dump sql".

Dans Supabase, ouvrir **SQL Editor > New query**.

1. Ouvrir le script SQL complet, copier tout son contenu, puis le coller dans l'éditeur.

2. Ouvrir **Table Editor** après le message de réussite.

Il doit y avoir cinq tables : `articles`, `pages`, `projects`, `skills` et `project_skills`.

Documentation Supabase - SQL Editor et tables

Vérifier si le schema est cohérent

Dans l'interface d'admin de Supabase, rendez-vous dans **Database ->Schema visualizer"** et vérifiez si tout vous semble correct.

> *Pro tip : Qui sera le premier à trouver le manquement ?*

Vérifier les clés de la table de liaison

Dans **SQL Editor**, examiner les contraintes créées :

```
select conname, contype
from pg_constraint
where conrelid = 'public.project_skills'::regclass
order by conname;
```

Attendre trois lignes : une clé primaire (`p`) et deux clés étrangères (`f`). Les noms exacts peuvent varier.

Aucune relation n'est saisie comme simple texte dans `projects` . Une ligne de `project_skills` contient deux identifiants existants.

Documentation PostgreSQL - Contraintes

III. Renseigner les contenus et contrôler l'accès

Saisir un premier article

Dans **Table Editor > articles**, cliquer **Insert row**.

CHAMP	VALEUR DE DÉPART
`title`	`Pourquoi TypeScript strict`
`slug`	`typescript-strict`
`excerpt`	`Un exemple avec typage du portfolio.`
`content`	`# TypeScript strict` puis un paragraphe
`cover_url`	laisser vide
`published_at`	laisser vide

Ne pas saisir `id` ni `created_at` : la base les génère. Cliquer **Save**. La ligne existe en base, mais reste masquée aux visiteurs.

Documentation Supabase - Charger des données

Ajouter une image de couverture

Les images vivent dans **Storage**, pas dans une colonne PostgreSQL.

1. Dans **Storage**, cliquer **New Bucket** et nommer `portfolio-images`.
2. Rendre ce bucket **Public**, puis créer le bucket.
3. L'ouvrir, cliquer **Upload File** et choisir une image du portfolio.
4. Copier son URL dans `cover_url` et la décrire dans `cover_alt`.

Pour un fichier nommé `cover.webp`, l'URL a cette forme :

```
https://<PROJECT_REF>.supabase.co/storage/v1/object/public/portfolio-images/cover.webp
```

Ouvrir l'URL dans un onglet privé : l'image doit s'afficher. Un fichier d'un bucket public reste accessible même si l'article est brouillon. N'y téléverser que des images destinées à être publiques.

> *Vous pouvez récupérer l'URL grâce au bouton "Get URL", dans le panel de droite, lorsque vous cliquez sur une image.*

Documentation Supabase - Servir les images

TP : Compléter le portfolio

Renseigner `published_at` de l'article précédent à `now()`, puis enregistrer.

Le brouillon devient publiable.

Ajouter ensuite dans **Table Editor** :

- une page `about` avec titre, contenu et `published_at`
- un projet `portfolio` avec résumé, description et `published_at`
- deux compétences, par exemple `TypeScript` et `React`
- un second article sans `published_at` pour tester les brouillons

Dans `project_skills`, ajouter le couple d'`id` du projet et d'une compétence.

Les `id` se lisent dans les lignes des tables correspondantes.

Compter six lignes de contenu, dont un article brouillon. [Documentation Supabase - Lignes et relations](https://supabase.com/docs/guides/database/tables

Comprendre clé publique et RLS

Une clé **publishable** sert d'identification publique et peut être visible dans le front.

Une clé **secret** contourne RLS et reste exclusivement côté serveur.

RLS, ou sécurité par ligne, décide quelles emtrées de la base chaque rôle peut lire.

`anon` représente un visiteur sans connexion Supabase.

Le script active RLS sur les cinq tables avant d'autoriser la lecture.

```
Table Editor : propriétaire de la base, saisie des contenus.
API avec clé publishable : droits SQL et politiques RLS.
```

Documentation Supabase - Clés API

Lire les politiques de publication

Le script crée une "policy" `SELECT` par table.

Pour `articles`, `pages` et `projects`, elle exige `published_at`.

Dans **SQL Editor**, inspecter les cinq politiques :

```
select tablename, policyname, qual
from pg_policies
where schemaname = 'public'
order by tablename;
```

> *Cherchez les entrées `published_at IS NOT NULL` dans les politiques des trois tables de contenu.*

Documentation Supabase - RLS

Contrôler la lecture seule

`revoke all` retire les anciens droits.

`grant select` rétablit seulement la lecture.

La politique RLS filtre ensuite les lignes accessibles.

Dans **SQL Editor**, vérifier les droits du rôle `anon` :

```
select
  has_table_privilege('anon', 'public.articles', 'SELECT') as can_read,
  has_table_privilege('anon', 'public.articles', 'INSERT') as can_write;
```

La même règle s'applique aux quatre autres tables.

Documentation Supabase - Droits et RLS

Vérifier les cinq tables

Contrôler que RLS est active :

```
select tablename, rowsecurity
from pg_tables
where schemaname = 'public'
order by tablename;
```

Attendre `true` pour `articles`, `pages`, `projects`, `skills` et `project_skills`. Vérifier ensuite que chaque table possède une politique `SELECT`.

```
select tablename, policyname, cmd from pg_policies where schemaname = 'public';
```

Documentation Supabase - Vérifier les tables

IV. Interroger l'API REST

Retrouver l'URL et la clé publique

Ouvrir **Connect** ou **Settings > API Settings** dans le projet.

Copier **Project URL** et la clé **publishable** (`sb_publishable_...`).

```
Project URL : https://<PROJECT_REF>.supabase.co
Route REST  : https://<PROJECT_REF>.supabase.co/rest/v1/articles
En-tête     : apikey: <PUBLISHABLE_KEY>
```

Ne jamais utiliser la clé **secret** ou la connexion PostgreSQL dans le front. Si la Data API a été désactivée, l'activer dans **Integrations > Data API** et exposer les tables `public` prévues pour le portfolio.

Repérer l'URL sans afficher la clé dans une capture de rendu. Documentation Supabase - Première route REST

Lire les articles avec `curl` et `fetch`

```
curl -i 'https://<PROJECT_REF>.supabase.co/rest/v1/articles?select=id,title,slug,excerpt' \
  -H 'apikey: <PUBLISHABLE_KEY>'
```

```
const url =
  "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=id,title,slug,excerpt";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const response = await fetch(url, { headers: { apikey: key } });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const articles: unknown = await response.json();
console.log(articles);
```

> *Que constatez-vous ?*

Documentation Supabase - Requête REST

Prouver que la lecture seule fonctionne

Essayer un ajout avec la même clé publique depuis la console du navigateur :

```
const url = "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const response = await fetch(url, {
  method: "POST",
  headers: { apikey: key, "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Test", slug: "test", excerpt: "Interdit" }),
});
console.log(response.status, await response.text());
```

Pour le moment, le tableau de bord reste le seul outil de saisie.

Documentation Supabase - Sécuriser l'API

Sélectionner, trier et limiter

Les paramètres de l'URL contrôlent la réponse :

```
const url =
  "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=title,slug,excerpt&order=created_at.desc&limit=5";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const response = await fetch(url, { headers: { apikey: key } });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const articles: unknown = await response.json();
console.log(articles);
```

`select` choisit les champs. `order` trie. `limit` borne la liste. La politique RLS reste appliquée avant le retour des lignes. Ajouter un troisième article publié dans Table Editor, puis relancer. Les titres doivent apparaître du plus récent au plus ancien. Documentation PostgREST - Filtrage et pagination

Récupérer un article par son slug

Le filtre `eq.` exprime une égalité sur une colonne :

```
const url =
  "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=title,slug,content&slug=eq.typescript-strict";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const response = await fetch(url, { headers: { apikey: key } });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const articles: unknown = await response.json();
console.log(articles);
```

La réponse reste un tableau. Il contient zéro ou un article, car `slug` possède une contrainte unique. Changer le slug demandé. Un slug absent doit produire `[]`. Un slug de brouillon doit également produire `[]` grâce à RLS. Documentation PostgREST - Filtres

Lire les pages, projets et compétences

Chaque table possède sa propre route REST :

```
const baseUrl = "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const routes = [
  "/pages?select=title,slug,content&slug=eq.about",
  "/projects?select=title,slug,summary&order=created_at.desc",
  "/skills?select=name,category,level&order=name.asc",
];

for (const route of routes) {
  const response = await fetch(baseUrl + route, { headers: { apikey: key } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  console.log(
    `Route: ${route}\n`,
    `Réponse: ${JSON.stringify(await response.json())}\n`,
  );
}
```

> *Pour lire les compétences d'un projet, interrogez aussi `project_skills` avec `project_id=eq.<ID>`, puis utiliser `skill_id`.*

Documentation Supabase - Routes REST

Charger les compétences d'un projet

PostgREST suit les clés étrangères de `project_skills` et expose la relation dans `select` :

```
const baseUrl = "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1";
const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";
const url = `${baseUrl}/projects?select=title,slug,skills(name)&slug=eq.portfolio`;
const response = await fetch(url, { headers: { apikey: key } });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
console.log(await response.json());
```

Documentation Supabase - Relations imbriquées

Préparer le front Vite

À la racine du portfolio React, créer `.env.local` :

```
VITE_SUPABASE_URL=https://<PROJECT_REF>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<PUBLISHABLE_KEY>
```

Remplacer les chevrons par les valeurs de **Connect**. Ces deux valeurs sont publiques : RLS protège les données. Ne jamais ajouter une clé **secret** dans une variable `VITE_` .

```
git check-ignore .env.local
```

La commande doit afficher `.env.local` . Sinon, l'ajouter à `.gitignore` . Documentation Supabase - React et variables Vite

Appeler REST depuis le portfolio

Dans une application Vite, seule la clé **publishable** porte le préfixe `VITE_`. Le service construit l'URL et vérifie la réponse HTTP :

```
const baseUrl = import.meta.env.VITE_SUPABASE_URL;
const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;
const url = new URL("/rest/v1/articles", baseUrl);
url.searchParams.set("select", "id,title,slug,excerpt");
const response = await fetch(url, { headers: { apikey: key } });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const articles: unknown = await response.json();
console.log(articles);
```

Valider `articles` avant de l'afficher dans un composant. Une variable `VITE_` est visible dans le navigateur. Documentation Supabase - Clés API

Contrôler les erreurs avant l'interface

- `200` avec `[]` : vérifier `published_at` et la politique `SELECT`.
- `401` : vérifier l'en-tête `apikey` et la clé publishable.
- `403` ou erreur `42501` : vérifier les droits SQL et RLS.
- `404` : vérifier l'URL, le nom de table et l'exposition Data API.

Quand les cinq routes et la restriction des brouillons fonctionnent, brancher les listes et pages de détail du portfolio sur ces réponses.

Documentation Supabase - Sécuriser l'API